Multi-Relay Assisted Computation Offloading for Multi-Access Edge Computing Systems With Energy Harvesting

Molin Li, Xiaobo Zhou, *Senior Member, IEEE*, Tie Qiu, *Senior Member, IEEE*, Qinglin Zhao, *Senior Member, IEEE*, and Keqiu Li, *Senior Member, IEEE*

***Abstract*—In multi-access edge computing systems with energy harvesting (MEC-EH), the mobile devices are empowered with unstable energy harvested from renewable energy sources. To prolong the life of mobile devices, as many computation-intensive tasks as possible should be offloaded to the MEC server. However, when the system states of mobile device and MEC server are unstable, e.g. poor communication channel conditions, a great number of tasks will be executed locally, leading to a long execution time. Even worse, some tasks may be dropped due to low energy levels. To address this problem, in this paper, we propose a multi-relay assisted computation offloading framework for MEC-EH systems. In this framework, a computation task can be executed by offloading to the MEC server with the help of multiple relay nodes, such as the neighboring nodes. We introduce execution cost as a performance metric to incorporate both the task execution time and task failure. We then develop a low-complexity online algorithm, namely MRACO algorithm, to minimize the average execution cost. MRACO algorithm can select the optimal execution strategy for each task from (1) executing the task locally, (2) offloading it to the MEC server directly, (3) offloading it to the MEC server with the help of the most suitable neighboring nodes, and (4) simply dropping it. Moreover, we also develop an algorithm for selecting the suitable neighboring devices to act as relays and determining the optimal task splitting ratio between them. Finally, performance evaluation shows that the proposed MRACO algorithm greatly outperforms the benchmarks in terms of both average execution time and task drop rate.**

***Index Terms*—Multi-access edge computing, computation offloading, energy harvesting, multi-relay.**

I. Introduction

With the rapid development of information and communication technologies, the mobile device can provide more and more content and service, which has been an important part of our daily life. Meanwhile, the demand for reliable and high-performance computing capabilities is also increasing [1]–[3]. Mobile cloud computing (MCC) is a general solution that can provide powerful computing capability for mobile devices [4]. However, the high transmission delay due to backbone network congestion becomes a big problem in time-sensitive scenarios. In the era of 5G, multi-access edge computing (MEC), which brings computation and storage resources to the edge of mobile network [5], empowers mobile devices with the ability to run the computation-intensive applications while meeting strict delay constraints [6], [7]. Powerful computation capacity and close geographic proximity will reduce computation delay and communication delay, respectively [8]. As a result, MEC has attracted significant attention from both academia and industry in recent years.

In MEC systems, the total energy of the conventional battery-powered mobile devices is limited. Even a powerful MEC system still cannot deliver its full performance when faced with energy-draining mobile devices. In other words, the computation tasks will be dropped, and the application cannot work normally after the battery energy is drained. Worse still, several mobile devices typically cannot be recharged repeatedly (e.g. in the wireless sensor networks), and others cannot use high-capacity chemical batteries due to hardware and size limitations [9], [10]. To solve this problem, we can integrate the energy harvesting (EH) technique into the MEC system. The energy harvesting technique can capture ambient recycling energy (e.g. Solar, wind, and RF energy) and store them in a battery for further use. In this way, the MEC system with EH (MEC-EH) can achieve stable and powerful computation capability [11], [12].

In the MEC-EH system, the energy harvested from the environment is unstable and precious. Therefore, the mobile device will first ensure that the battery power is not depleted to keep the equipment functioning normally. The mobile devices can offload tasks to the MEC server to reduce execution time when the energy and latency condition is satisfied. Wireless communication channels in MEC-EH system are often highly stochastic due to the presence of noise interference and signal fading in the natural environment [13]. When the communication channel between the mobile device and the MEC server is poor, the tasks will be executed locally, leading to an increase in execution time. Moreover, when the battery level is low, the tasks may even be

Manuscript received April 20, 2021; revised June 22, 2021; accepted August 16, 2021. Date of publication August 30, 2021; date of current version October 15, 2021. This work was supported in part by the National Key Research and Development Program of China under Grant 2019YFB2102404, in part by NSFC Joint Funds under Grants U1701263, U1836214, and U2001204, in part by the National Natural Science Foundation of China under Grants 62072330 and 61872451, and in part by Science and Technology Development Fund, Macau SAR under Grant 0062/2020/A2. The review of this article was coordinated by Prof. Swades De. *(Corresponding author: Xiaobo Zhou.)*

Molin Li, Xiaobo Zhou, Tie Qiu, and Keqiu Li are with the Tianjin Key Laboratory of Advanced Networking, College of Intelligence and Computing, Tianjin University, Tianjin, China (e-mail: molinchn@tju.edu.cn; xiaobo.zhou@tju.edu.cn; qiutie@ieee.org; keqiu@tju.edu.cn).

Qinglin Zhao is with the Faculty of Information Technology, Macau University of Science and Technology, Avenida Wei Long, Taipa, Macau, China (e-mail: zqlict@hotmail.com).

Digital Object Identifier 10.1109/TVT.2021.3108619

dropped to avoid depletion of the energy of the mobile device. In [14], the mobile device can offload its tasks to the MEC server via the help of a neighboring node (i.e., relay node) if the channel quality of the direct link (i.e., the link between the mobile device and the MEC server) falls under a certain threshold. In practice, there are usually multiple neighboring nodes around the mobile devices that can help combat the unstable system state and further improve the system performance, but these neighboring nodes are not fully utilized.

In this paper, we consider a more general scenario with multiple relays and propose a multi-relay assisted computation offloading framework for the MEC-EH system. In our framework, a computation task can be executed by offloading to the MEC server with the help of multiple relay nodes. More specifically, when the transmission delay is too high to offload tasks in MEC-EH system, the mobile device can choose some of its neighboring nodes that have better communication channels and ask them to act as relay nodes to establish alternative routes to the MEC server. In this way, more tasks will be offloaded to the MEC server, thereby significantly reducing the average execution time and task failure rate of tasks. Our major contributions are detailed below:

- We consider a MEC-EH system and propose the multi-relay assisted computation offloading framework, which allows computing tasks to be offloaded to MEC server by multi-relay assisted routes. We use execution cost as a performance metric to incorporate both the task execution time and task failure. Our target is to reduce the average cost of the MEC-EH system by selecting the optimal execution strategy. The mobile device will get a lower execution time while maintaining energy self-sufficient.
- We formulate the execution cost minimization problem as a high-dimensional Markov decision problem, and we propose a low-complexity online algorithm, namely multi-relay assisted computation offloading (MRACO) algorithm, to solve this problem. The algorithm selects the optimal execution strategy based on the current system state, where the strategies including executing locally, offloading to MEC server directly, offloading to MEC server via multi-relay assisted offloading, or drop the computing tasks.
- We also develop an algorithm for selecting the suitable neighboring devices to act as relays, when the strategy of offloading to MEC server via multi-relay assisted routes is selected. To obtain lower latency, we also derive the optimal task splitting ratio between multiple relays.

The rest of this paper is organized as follows. Section II presents related works. In Section III, we introduce the MEC-EH system model considered in this paper. The problem formulation is presented in Section IV, and the MRACO algorithm is detailed in Section V. We show the simulation results in Section VI. Finally, we conclude this paper in Section VII.

II. Related Works

In the MEC system, computation offloading is an important research field. Many researchers focus on optimizing different aspects of the computation offloading problems. A single-user MEC system is considered in [15] and the authors aim to minimize the average execution latency of all tasks under power constraints by deriving the optimal resource allocation solution. In [16], the authors consider a multi-user MEC system and propose an efficient algorithm to minimize the overall delay while satisfying the computation requirements of mobile devices. The authors in [8] propose a learning-based algorithm to improve task assignment efficiency, which aims to minimize computing energy cost. In [17], the authors introduce the users' task offloading gains as the performance metric, which incorporate the reduction in task completion time and energy consumption. Then they propose an algorithm to make joint task offloading and resource allocation decision to maximize the users' task offloading gains.

The MEC-EH system has also attracted attention from both academia and industry over the past several years. In [18], the energy from the battery and EH device will be used to power the edge server. However, unpredictability energy makes it challenging to deliver a high quality of service (QoS). Therefore, authors propose a learning-based resource management algorithm to determine the optimal offloading strategy and autoscaling actions. In [19], authors consider a green MEC (GMEC) system enabled dynamic adaptive video streaming over HTTP (DASH). The MEC server is powered by EH devices and grid electricity. Authors propose a greedy-based algorithm to control the video bitrate that can jointly optimize the quality of experience (QoE) and backhaul traffic. Besides, EH devices can be used to power wireless devices. In [20], authors investigate a MEC-EH system and develop an effective computation offloading strategy. A low-complexity online algorithm is proposed in this paper, namely, the Lyapunov optimization-based dynamic computation offloading (LODCO) algorithm. However, in [20], the computation task can only be executed locally or offloaded to the MEC server. When the MEC-EH system is in a bad state, the performance of the system will be poor. In [21], authors consider a multi-user MEC network with wireless power transfer (WPT). Then they propose an online algorithm that optimally adapts task offloading decisions and wireless resource allocations to the time-varying wireless channel conditions.

A large number of connections are expected to be heterogeneous, demanding higher data transmission rates, reduced execution time, enhanced system capacity, and superior throughput. In common research scenarios, mobile devices offload their tasks directly to MEC servers. These researches did not consider that a mobile device can act as a relay and help other devices to communicate with MEC servers. An emerging facilitator of the upcoming high data rates demanding next generation networks (NGNs) is device-to-device (D2D) communication [22]. The cooperative relay is an important part of D2D communication. A cooperative relay [23] highlights the cooperative transmission between multiple users. The first cooperative relay communication framework is proposed in [24], which improves the performance of both D2D links and cellular links. In [25]–[27], the authors consider the scenario where a mobile device can offload its task via the relay devices, and propose different algorithms to minimize latency and energy consumption. However, the relay devices are powered by batteries and the total energy is limited.

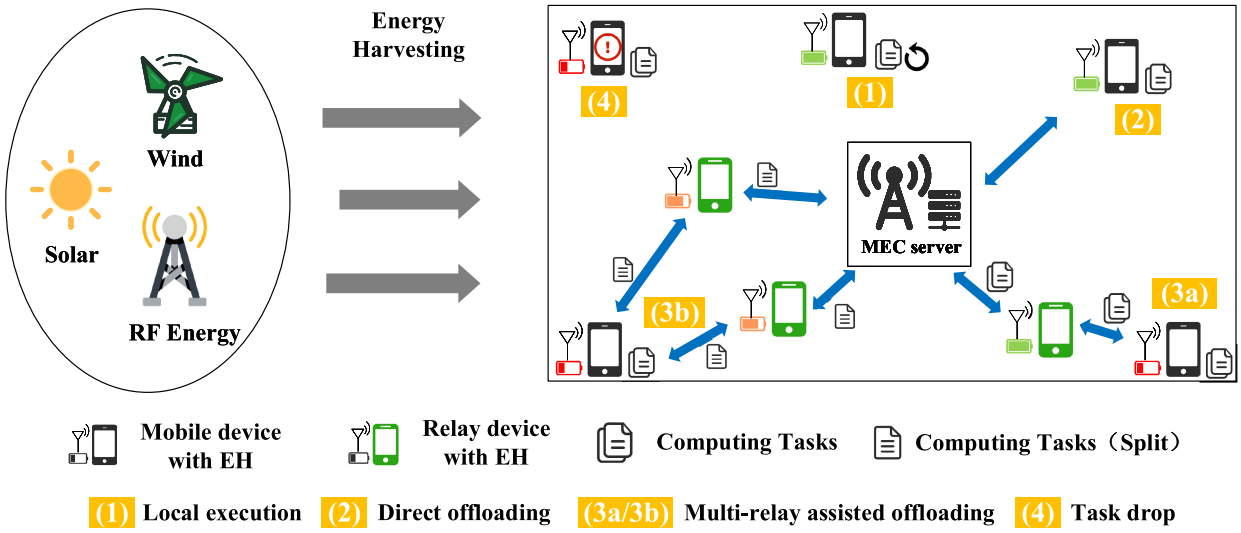


Fig. 1. Overview of the MEC-EH system.

Different from previous work, in this paper, we consider a MEC-EH system, where the mobile devices are EH-enabled. The multi-relay assisted offloading is allowed so that mobile devices can offload its tasks via selected multiple relays. We also propose the MRACO algorithm to decide the optimal offloading strategies, including relay selection, task splitting, and transmission power allocation, to minimize the average task execution time while satisfying the self-sufficiency of energy.

III. System Model

In this section, firstly, we introduce the MEC-EH system, including energy harvesting system and user mobility model. Then, we also show how to calculate the task execution time and energy consumption when the task is executed locally, directly offloaded to the MEC server, or offloaded to the MEC server with the help of multiple relays.

A. MEC-EH System

We consider a MEC-EH system consists of multiple mobile devices and a base station, as shown in Fig. 1. A MEC server is deployed at the base station which provides powerful computing capability for the mobile devices. Each mobile device is equipped with a EH module that can harvest ambient recycling energy (e.g. solar, wind, and RF energy) and store them in battery. We divide time into discrete time slots, each of which has a duration of Δt. Let t denote the index of time slots, where $t \in \mathcal{T}, \mathcal{T} = \{0, 1, 2, \ldots, T\}$.

In this paper, we focus on the task execution process of one mobile device. At the beginning of time slot t, a computation task may arrive at the mobile device and then be executed. There are four optional task execution strategies: (1) the mobile device executes the task locally, (2) the mobile device offloads the task to the MEC server directly, and receives the result returned from the MEC server, (3) the mobile device offloads the task to the MEC server via the help of its neighboring devices (in other words, the neighboring devices act as relays), and also receives the result returned from the MEC server, and (4) just drop the task if the deadline requirement cannot be satisfied, as shown in Fig. 1. The former three strategies are referred to as *local execution*, *direct offloading* and *multi-relay assisted offloading*, respectively. At the arrival of the task, the mobile device rely on offloading algorithms to decide how to execute this task, i.e., which task execution strategy should be selected, by taking the battery level, task execution time and energy consumption into account. For ease of reference, the important symbols used in this article are shown in the TABLE I.

B. Energy Harvesting Model

Since time is non-continuous in the model of this paper, the process of energy collection is also non-continuous. Let E_{H}^t represent the energy arriving at the t-th time slot, and we assume that E_{H}^t is i.i.d.[1] and satisfies

$$E_{\mathrm{H}}^t \leq E_H^{\max}, \tag{1}$$

where $E_H^{\max}$ is the maximum value of the energy arriving at each time slot. To match the actual energy conversion efficiency, we assume that part of E_{H}^t, denoted as e^t, will be harvested and stored in the battery at each time slot. e^t will be available from the next time slot, and we therefore have

$$0 \leq e^t \leq E_{\mathrm{H}}^t. \tag{2}$$

We denote the energy level of the battery in the t-th time slot as B^t and set $B^0 = 0$, then for each time slot, the battery energy level satisfies

$$B^t \geq 0. \tag{3}$$

For simplicity, we ignore the energy consumption outside of task execution such as the operating system. Therefore, we have

$$B^{t+1} = B^t + e^t - E_{\mathrm{exec}}^t, \tag{4}$$

[1]Although the model and assumptions is simple, it can capture the stochastic and intermittent nature of energy harvesting process [20], [28], [29].

TABLE I
SUMMARY OF SYMBOLS

Symbol	Definition
t	Time slot index
Δt	Length of time slot
L^t	Task bit size
t_{d}^t	Task execution deadline
P_{H}	Energy harvesting power
B^t	Battery level
P_{new}	The probability of visiting a new location
P_{old}	The probability of visiting an old location
f^t	CPU frequency
$f_{\mathrm{CPU}}^{\max}$	Maximum CPU frequency
ω	Bandwidth
σ	Noise power
p_0^t	Transmission power when offload directly
$p_{1,i}^t, p_{2,i}^t$	Transmission power when offload via i-th relay device
$p_{\mathrm{trans}}^{\max}$	Maximum transmission power
$E_{\max}$	Maximum energy consumption per task
M^t	Number of nearby relay devices
M_{relay}^t	Number of finally selected relay devices
y_i	Indicate whether i-th relay is selected
ρ_i	Task split ratio of i-th relay device
ϕ	Penalty factor
E_{s}^t	Energy consumption for direct offloading
E_{m}^t	Energy consumption for mobile execution
E_{r}^t	Energy consumption when using relay-assisted offloading
E_{exec}^t	Energy consumption
T_{s}^t	Execution time for direct offloading
T_{m}^t	Execution time for mobile execution
T_{r}^t	Execution time when using relay-assisted offloading
T_{exec}^t	Task execution time

where E_{exec}^t is the energy consumption of task execution at time slot t, which will be detailed in Section III-F.

C. User Mobility Model

To be more realistic, we consider the case that all the users are constantly moving. Specifically, in the MEC-EH system considered in this paper, we use the Individual Mobility Model (IMM) [30] to describe user mobility. Differing from traditional user mobility models, such as the random waypoint (RWP) model [31], IMM considers human tendency behavior. In IMM model, users tend to visit their frequently visited locations, which is more realistic [32]. According to IMM, at time slot t, the i-th user located at place (a_i^t, b_i^t) chooses a destination $(a_i^{t'}, b_i^{t'})$ and starts moving towards the destination at speed v_i^t. When the user arrives at the destination, he will stop moving for a random period, then continue to choose another destination and move towards the new destination at a new speed.

Since users may go to the same destination multiple times during a certain period (i.e., frequent visited places), the probability of visiting a new location that has never been visited is defined as

$$P_{\mathrm{new}}^t = \beta (N_{\mathrm{hist}}^t)^{-\gamma}, \tag{5}$$

where N_{hist}^t is the number of visited locations, and $0 < \beta \leq 1$ and $\gamma > 0$ are fixed parameters that are related with user mobility habits. The probability of visiting an old location that has been visited in the past is

$$P_{\mathrm{old}}^t = 1 - \beta (N_{\mathrm{hist}}^t)^{-\gamma}. \tag{6}$$

It can be found that as the user visits more locations, i.e., the value of N_{hist}^t becomes larger, the probability of the user visiting a new locations becomes smaller while the probability of the user visiting an old location becomes larger.

D. Local Execution

When the computing tasks need to be completed, there are several ways to execute the tasks. Specifically, we can execute them on the mobile device, by using the CPU and with another source on the mobile device. In addition, we can directly offload them to the MEC server, using the powerful CPU and greater memory resources for the tasks.

We denote the probability of the mobile device generating a computing task at each time slot is η. In addition, we denote the computing task with two-tuples (L^t, t_{d}^t), where L^t is the input size of the task and t_{d}^t is the execution time deadline of the task. Particularly, t_{d}^t indicates the maximum execution time. Since we focus on the time sensitive application, the execution time must be smaller than the execution time deadline. Otherwise, the task will fail to execute and be dropped.

Let X denote the number of CPU cycles required for mobile device to process 1-bit computing task, then $L^t X$ cycles are required to process a computing task of size L^t. We assume that the CPU on the mobile device is controlled by DVFS technology [13], and the CPU frequencies scheduled for task (L^t, t_{d}^t) is f^t. When executed locally, the execution time and energy consumption of a computing task are

$$T_{\mathrm{m}}^t = L^t X (f^t)^{-1}, \tag{7}$$

and

$$E_{\mathrm{m}}^t = \kappa L^t X (f^t)^2, \tag{8}$$

respectively, where (8) can be used to calculate the energy consumption generated by the CPU of mobile devices, and κ is a parameter that depends on the chip architecture [33]. Due to the limited CPU computing power, we define the maximum CPU frequency as $f_{\mathrm{CPU}}^{\max}$, i.e., $f^t \leq f_{\mathrm{CPU}}^{\max}$.

E. Direct Offloading

When executed on the MEC server, the task (L, t_{d}) should be transmitted to the MEC server first. Let p_0^t and $p_{\mathrm{trans}}^{\max}$ denote the transmission power of the mobile device at the t-th time slot and the maximum transmission power, respectively. Therefore we have $p_0^t \leq p_{\mathrm{trans}}^{\max}$. The maximum transmission rate of the channel at the t-th time slot is

$$r^t = w \log_2 \left(1 + \frac{h^t p_0^t}{\sigma^2}\right), \tag{9}$$

where w is the channel bandwidth, σ is the noise power, and h^t is the channel gain.[2]

[2] h^t is exponentially distributed with mean $g_0(d_0/d)^4$, where $g_0 = -40$ dB, $d_0 = 1$ m, and d is the distance between the mobile device and the MEC server.

Similar to [34], [35], since MEC servers are usually equipped with high-performance CPUs, we can ignore the task execution time for convenience. Also, when the results of the task computation are returned, the transmission time can be neglected as its small size. Therefore, the execution time and energy consumption of the tasks offloaded to the MEC server directly are

$$T_{\mathrm{s}}^t = \frac{L^t}{r^t}, \tag{10}$$

and

$$E_{\mathrm{s}}^t = p_0^t \cdot T_{\mathrm{s}}^t = p_0^t \cdot \frac{L^t}{r^t}, \tag{11}$$

respectively.

F. Multi-Relay Assisted Offloading

In the multi-relay assisted offloading strategy, the mobile device perform task offloading with the help of its nearby relay devices.[3] These relay devices will satisfy the conditions including energy sufficient and be idle for a period of time. Denote the number of nearby relay devices as M^t. We select M_{relay}^t relay devices and split the task into M_{relay}^t parts, which satisfies

$$M_{\mathrm{relay}}^t = \min\{M^t, K^t\}, \tag{12}$$

where K^t is the maximum number that the task can be split into, and it usually depends on the task. Note that there are many kinds of tasks that can be split, such as convolution operations in DNN [37]. These relay devices will help to transmit the tasks to the MEC server. In this way, the mobile device could offload tasks with no need for a large amount of transmission energy consumption.

We use $\boldsymbol{y}^t = \{y_i^t | i = 1, 2, \ldots, M^t\}$ to denote relay selection, where $y_i^t = 1$ indicates the i-th relay is selected, $y_i^t = 0$ indicates the i-th relay is not selected. We have

$$\sum_{i=1}^{M^t} y_i^t = M_{\mathrm{relay}}^t. \tag{13}$$

We denote that after the splitting, the task size of the i-th device is

$$L_i^t = y_i^t \rho_i^t \cdot L^t, \tag{14}$$

where $i = 1, 2, \ldots, M^t$, ρ_i^t is the task split ratio, and part of the task (with size $\rho_i^t \cdot L$) will be transmitted to the i-th idle device. There is $\boldsymbol{\rho}^t = \{\rho_i^t | i = 1, 2, \ldots, M^t\}$. In addition, ρ_i^t satisfies the condition

$$\sum_{i=1}^{M^t} y_i^t \rho_i^t = 1. \tag{15}$$

When $M_{\mathrm{relay}}^t = 1$, there is only one idle device that can be offloaded, and thus the task does not need to be split. We will transmit all of the task input to the idle device, and then the idle device will transmit the task to the MEC server.

[3]By utilizing 5 G mmWave and beamforming techniques, the mobile device can communicate with relay devices in parallel without signal interference [36].

We denote that the execution time and energy consumption from the computing device to the i-th relay device in t-th time slot are $T_{1,i}^t$ and $E_{1,i}^t$, respectively. The execution time and energy consumption from the i-th relay device to the MEC server in t-th time slot are $T_{2,i}^t$ and $E_{2,i}^t$, respectively. We have

$$T_{1,i}^t = \frac{\rho_i^t L^t}{r_{1,i}^t},\ T_{2,i}^t = \frac{\rho_i^t L^t}{r_{2,i}^t}, \tag{16}$$

and

$$E_{1,i}^t = p_{1,i}^t \cdot T_{1,i}^t,\ E_{2,i}^t = p_{2,i}^t \cdot T_{2,i}^t, \tag{17}$$

where $r_{1,i}^t$ and $p_{1,i}^t$ are transmission rate and transmission power from mobile device to i-th relay device, $r_{2,i}^t$ and $p_{2,i}^t$ are same but from relay device to MEC server. $r_{1,i}^t$ and $r_{2,i}^t$ can be calculated by

$$r_{1,i}^t = w \log_2\left(1 + \frac{h_{1,i} p_{1,i}^t}{\sigma^2}\right), r_{2,i}^t = w \log_2\left(1 + \frac{h_{2,i} p_{2,i}^t}{\sigma^2}\right), \tag{18}$$

$h_{1,i}$ and $h_{2,i}$ are channel power gain from the mobile device to the i-th relay device and that from the i-th relay device to the MEC server, respectively.

The total execution time and energy consumption of multi-relay assisted offloading are

$$T_{\mathrm{r}}^t = \max_{i \in [1, M^t]} \left[(T_{1,i}^t + T_{2,i}^t) \cdot y_i^t\right], \tag{19}$$

and

$$E_{\mathrm{r}}^t = \sum_{i=1}^{M^t} (E_{1,i}^t \cdot y_i^t), \tag{20}$$

respectively.

When the computing task is subject to execution time or energy conditions and cannot be successfully completed regardless of which execution strategy is selected, the task will be dropped. In this case, the execution of the task will fail, although there is no energy consumption and execution time. We use $\boldsymbol{I}^t = [I_{\mathrm{m}}^t, I_{\mathrm{s}}^t, I_{\mathrm{r}}^t, I_{\mathrm{d}}^t]^T$ to indicate the task execution strategy, where $I_{\mathrm{m}}^t, I_{\mathrm{s}}^t, I_{\mathrm{r}}^t, I_{\mathrm{d}}^t \in \{0, 1\}$. When a computing task arrives, one execution strategy will be selected, therefore we have

$$I_{\mathrm{m}}^t + I_{\mathrm{s}}^t + I_{\mathrm{r}}^t + I_{\mathrm{d}}^t = 1, \quad t \in \mathcal{T}. \tag{21}$$

Note $I_{\mathrm{m}}^t = 1$, $I_{\mathrm{s}}^t = 1$ and $I_{\mathrm{r}}^t = 1$ denotes *local execution*, *direct offloading* and *multi-relay assisted offloading*, respectively. $I_{\mathrm{d}}^t = 1$ indicates that the task is dropped.

According to (8), (11) and (20), E_{exec}^t in (4) can be formulated as

$$E_{\mathrm{exec}}^t = E_{\mathrm{m}}^t \cdot I_{\mathrm{m}}^t + E_{\mathrm{s}}^t \cdot I_{\mathrm{s}}^t + E_{\mathrm{r}}^t \cdot I_{\mathrm{r}}^t, \tag{22}$$

and the execution time of t-th time slot can be calculated by

$$T_{\mathrm{exec}}^t = T_{\mathrm{m}}^t \cdot I_{\mathrm{m}}^t + T_{\mathrm{s}}^t \cdot I_{\mathrm{s}}^t + T_{\mathrm{r}}^t \cdot I_{\mathrm{r}}^t. \tag{23}$$

IV. Problem Formulation

In the MEC-EH system, the primary goal is to minimize the average task execution time while guaranteeing the energy of the mobile device not be exhausted. However, when task failure

happens, the task is dropped and no task execution time is incurred. To incorporate task failure, we introduce execution cost as a performance metric, which is denoted as

$$cost^t = T^t_{\text{exec}} + \phi \cdot I^t_{\text{d}}. \tag{24}$$

Here, ϕ is the penalty of task failure. Thus the objective of the MEC-EH system considered can be formulated as

$$\mathcal{P}_1: \min_{\boldsymbol{I}^t, f^t, \boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t, e^t} \lim_{T\to+\infty} \frac{1}{T} E\left[\sum_{t=0}^{T-1} cost^t\right]$$

$$s.t.\ (2), (3), (13), (15), (21)$$

$$E^t_{\text{exec}} \in \{0\} \cup [E_{\min}, E_{\max}], \quad t \in \mathcal{T} \tag{25}$$

$$T^t_{\text{exec}} \le t_{\text{d}}, \quad t \in \mathcal{T} \tag{26}$$

$$0 \le p^t_0, p^t_{1,i}, p^t_{2,i} \le p^{\max}_{\text{trans}} \cdot \left(I^t_{\text{s}} + I^t_{\text{r}}\right), t \in \mathcal{T} \tag{27}$$

$$0 \le f^t \le f^{\text{CPU}}_{\max} \cdot I^t_{\text{m}}, \quad t \in \mathcal{T} \tag{28}$$

$$M^t_{\text{relay}} = \min\{M^t, K^t\} \cdot I^t_{\text{r}}, \quad t \in \mathcal{T}, \tag{29}$$

where $\boldsymbol{p}^t = [p^t_0, p^t_{1,1}, p^t_{2,1}, \ldots, p^t_{1,M^t}, p^t_{2,M^t}]^T$, $\boldsymbol{y}^t = [y^t_1, y^t_2, \ldots, y^t_{M^t}]^T$, $\boldsymbol{\rho}^t = [\rho^t_1, \rho^t_2, \ldots, \rho^t_{M^t}]^T$.

Constraint (25) limits the energy consumption at t-th time slot by setting the minimum and maximum energy consumption. Similar to [20], we set a lower-bound energy consumption to avoid the time-varying battery level constraint. Otherwise, the problem will become complicated. We also prove that with the solution obtained from the proposed MRACO algorithm, this constraint is satisfied, as detailed in Section V-C. Constraint (26) indicates that the execution time of the task should be smaller than its deadline. Constraints (27) and (28) specify the limitations of the transmission power and the CPU frequency, respectively. Constraint (29) limits the number of relay devices.

V. Algorithm Design

A. Problem Transformation

In the MEC-EH system, the system state includes positions and battery levels of devices, channel state, and information of tasks. The action is composed of the execution strategy, CPU frequency, transmission power, relay selection, and task split ratio. Besides, current action only depends on the current system state. Therefore, $\mathcal{P}_1$ is a Markov decision process (MDP). However, it cannot be solved by traditional MDP algorithms due to state space explosion. Recently deep reinforcement learning (DRL) is widely used in solving this kind of problem, but the optimality of the DRL algorithms cannot be guaranteed. Moreover, DRL algorithms may spend a considerable amount of time in the training step. Instead, we utilize the Lyapunov optimization technique and transfer $\mathcal{P}_1$ to $\mathcal{P}_2$ as

$$\mathcal{P}_2: \min_{\boldsymbol{I}^t, f^t, \boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t, e^t} \tilde{B}^t\left(e^t - E_{\text{exec}}\right) + V cost^t$$

$$s.t.\ (2), (3), (13), (15), (21), (25) - (29),$$

where

$$\tilde{B}^t = B^t - \theta \tag{30}$$

Algorithm 1: MRACO Algorithm.

Input: $t = 0$, $B^0 = 0$, (L^t, t^t_{d}), E^t_{H}, h_t, mobile device position (a_0, b_0), base station position $(a_{\text{m}}, b_{\text{m}})$, relay devices positions (a_i, b_i),θ,V

Output: $\boldsymbol{I}^t$, f^t, $\boldsymbol{p}^t$, $\boldsymbol{y}^t$, $\boldsymbol{\rho}^t$

```
while system is running do
    a^t, b^t ← IMM(a^{t-1}, b^{t-1});
    e^t ← E^t_H · 1(B̃^t ≤ 0);
    B^{t+1} ← B^t + e^t − E^t_exec;
    B̃^t = B^t − θ;
    if task arrives then
        Set I_m = 1, I_s = I_r = I_d = 0;
        Obtain P*_m by solving P_m;
        Set I_s = 1, I_m = I_r = I_d = 0;
        Obtain P*_s by solving P_s;
        Set I_r = 1, I_m = I_s = I_d = 0;
        Obtain P*_r by solving P_r;
        Obtain P*_4 and {I^{t*}, f^{t*}, p^{t*}, y^{t*}, ρ^{t*}}
          according to (39);
    end
    t ← t + 1;
end
```

is the virtual energy queue. Here θ is the perturbation parameter that satisfies

$$\theta \ge \tilde{E}_{\max} + V\phi \cdot E^{-1}_{\min}, \tag{31}$$

where $0 < V < +\infty$ is the control parameter, and

$$\tilde{E}_{\max} = \min\{\max\{\kappa L^t X (f^{\max}_{\text{CPU}})^2, p^{\max}_{\text{trans}} t\}, E_{\max}\} \tag{32}$$

is the maximum energy consumption to execute next task.

As the objective function of $\mathcal{P}_2$ can be decomposed into two uncorrelated parts as follows

$$\begin{aligned}&\tilde{B}^t\left(e^t - E_{\text{exec}}\right) + V cost^t \\ &= \tilde{B}^t e^t + \left(-\tilde{B}^t E_{\text{exec}} + V cost^t\right),\end{aligned} \tag{33}$$

we further decompose $\mathcal{P}_2$ into $\mathcal{P}_3$ and $\mathcal{P}_4$ as

$$\mathcal{P}_3: \min_{0 \le e^t \le E^t_{\text{H}}} \tilde{B}^t e^t,$$

and

$$\mathcal{P}_4: \min_{\boldsymbol{I}^t, f^t, \boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t} J^t\left(\boldsymbol{I}^t, f^t, \boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t\right)$$

$$s.t.\ (3), (13), (15), (21), (25) - (29),$$

where $J^t\left(\boldsymbol{I}^t, f^t, \boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t\right) = -\tilde{B}^t E_{\text{exec}} + V cost^t$. Once we obtain the solutions of $\mathcal{P}_3$ and $\mathcal{P}_4$, $\mathcal{P}_2$ is solved.

B. MRACO Algorithm

In this subsection, we present the proposed MRACO algorithm to solve $\mathcal{P}_3$ and $\mathcal{P}_4$. Since $\mathcal{P}_3$ is a linear programming problem, its optimal solution can be easily obtained as

$$e^{t*} = E^t_{\text{H}} \cdot \mathbf{1}(\tilde{B}^t \le 0), \tag{34}$$

where $\mathbf{1}(\cdot)$ is the indicator function, i.e., it will be assigned a value of 1 when the conditions are satisfied, and it will be assigned a value of 0 otherwise.

On the other hand, $\mathcal{P}_4$ can be further expanded as

$$J^t\left(\boldsymbol{I}^t, f^t, \boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t\right) = I_{\mathrm{m}}^t \cdot J_{\mathrm{m}}^t\left(f^t\right) + I_{\mathrm{s}}^t \cdot J_{\mathrm{s}}^t\left(p_0^t\right) + I_{\mathrm{r}}^t \cdot J_{\mathrm{r}}^t\left(\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t\right) + I_{\mathrm{d}}^t \cdot V\phi, \tag{35}$$

where

$$J_{\mathrm{m}}^t\left(f^t\right) = -\tilde{B}^t \cdot E_{\mathrm{m}} + V \cdot T_{\mathrm{m}}, \tag{36}$$

$$J_{\mathrm{s}}^t\left(p_0^t\right) = -\tilde{B}^t \cdot E_{\mathrm{s}} + V \cdot T_{\mathrm{s}}, \tag{37}$$

and

$$J_{\mathrm{r}}^t\left(\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t\right) = -\tilde{B}^t \cdot E_{\mathrm{r}} + V \cdot T_{\mathrm{r}} \tag{38}$$

corresponds to *local execution*, *direct offloading* and *multi-relay assisted offloading*, respectively. When a task arrives, we consider the following four cases.

- *Case 1:* $I_{\mathrm{m}} = 1, I_{\mathrm{s}} = I_{\mathrm{r}} = I_{\mathrm{d}} = 0$. In this case, the task is executed locally on the mobile device, therefore we have $\boldsymbol{p}^t = \mathbf{0}, \boldsymbol{y}^t = \mathbf{0}, \boldsymbol{\rho}^t = \mathbf{0}$ and $\mathcal{P}_4$ is equivalent to

$$\mathcal{P}_{\mathrm{m}}: \quad \min_{f^t} \ J_{\mathrm{m}}^t(f^t)$$
$$s.t.\ (3), (25), (26), (28).$$

- *Case 2:* $I_{\mathrm{s}} = 1, I_{\mathrm{m}} = I_{\mathrm{r}} = I_{\mathrm{d}} = 0$. In this case, the task is offloaded directly to the edge server, therefore we have $f^t = 0, p_{1,i}^t = p_{2,i}^t = 0, \boldsymbol{y}^t = \mathbf{0}, \boldsymbol{\rho}^t = \mathbf{0}$ and $\mathcal{P}_4$ is equivalent to

$$\mathcal{P}_{\mathrm{s}}: \quad \min_{p_0^t} \ J_{\mathrm{s}}^t(p_0^t)$$
$$s.t.\ (3), (25) - (27).$$

- *Case 3:* $I_{\mathrm{r}} = 1, I_{\mathrm{m}} = I_{\mathrm{s}} = I_{\mathrm{d}} = 0$. In this case, the task is executed by relay assisted computation offloading, therefore we have $f^t = 0, p_0^t = 0$ and $\mathcal{P}_4$ is equivalent to

$$\mathcal{P}_{\mathrm{r}}: \quad \min_{\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t} \ J_{\mathrm{r}}^t(\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t)$$
$$s.t.\ (3), (13), (15), (25) - (27), (29).$$

- *Case 4:* $I_{\mathrm{d}} = 1, I_{\mathrm{m}} = I_{\mathrm{s}} = I_{\mathrm{r}} = 0$. In this case, the task is dropped. $\mathcal{P}_4$ is equivalent to $\mathcal{P}_{\mathrm{d}}$.

Let $\mathcal{P}_{\mathrm{m}}^*$, $\mathcal{P}_{\mathrm{s}}^*$, $\mathcal{P}_{\mathrm{s}}^*$ and $\mathcal{P}_{\mathrm{d}}^*$ denote the optimal solution of *Case 1*, *Case 2*, *Case 3* and *Case 4*, respectively. Note that $\mathcal{P}_{\mathrm{m}}^*$ and $\mathcal{P}_{\mathrm{s}}^*$ can be obtained with the LODCO algorithm [20], and $\mathcal{P}_{\mathrm{r}}^*$ can be obtained using Algorithm 2 that is detailed in Section V-C. Since there is no energy consumption and execution time due to task failure in *Case 4*, we have $\mathcal{P}_{\mathrm{d}}^* = V\phi$. Thus the optimal solution of $\mathcal{P}_4$ is

$$\mathcal{P}_4^* = \min\{\mathcal{P}_{\mathrm{m}}^*, \mathcal{P}_{\mathrm{s}}^*, \mathcal{P}_{\mathrm{r}}^*, \mathcal{P}_{\mathrm{d}}^*\}, \tag{39}$$

and the optimal offloading strategy $\{\boldsymbol{I}^{t*}, f^{t*}, \boldsymbol{p}^{t*}, \boldsymbol{y}^{t*}, \boldsymbol{\rho}^{t*}\}$ can be obtained subsequently. The MRACO algorithm is summarized in Algorithm 1.

C. Solve Problem $\mathcal{P}_{\mathrm{r}}$

$\mathcal{P}_{\mathrm{r}}$ can be formulated as

$$\mathcal{P}_{\mathrm{r}}: \min_{\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t} -\tilde{B}^t \cdot \sum_{i=1}^{M^t} \left[\frac{p_{1,i}^t y_i^t \rho_i^t L^t}{r_{1,i}^t}\right] + V \cdot \max_{i \in [1, M^t]} \left[\frac{y_i^t \rho_i^t L^t}{r_{1,i}^t} + \frac{y_i^t \rho_i^t L^t}{r_{2,i}^t}\right]$$

$$s.t. \quad M_{\mathrm{relay}}^t = \min\{M^t, K^t\}, \quad t \in \mathcal{T} \tag{40}$$

$$\sum_{i=1}^{M^t} y_i^t = M_{\mathrm{relay}}^t, \quad t \in \mathcal{T} \tag{41}$$

$$\sum_{i=1}^{M^t} y_i^t \rho_i^t = 1, \quad t \in \mathcal{T} \tag{42}$$

$$\max_{i \in [1, M^t]} \left[\frac{y_i^t \rho_i^t L^t}{r_{1,i}^t} + \frac{y_i^t \rho_i^t L^t}{r_{2,i}^t}\right] \le t_{\mathrm{d}}, \quad t \in \mathcal{T} \tag{43}$$

$$\sum_{i=1}^{M^t} \left[\frac{p_{1,i}^t y_i^t \rho_i^t L^t}{r_{1,i}^t}\right] \in [E_{\min}, E_{\max}], \quad t \in \mathcal{T} \tag{44}$$

$$0 \le p_{1,i}^t, p_{2,i}^t \le p_{\mathrm{trans}}^{\max}, \quad t \in \mathcal{T}. \tag{45}$$

As can be seen, $\mathcal{P}_{\mathrm{r}}$ is a multivariable optimization problem which cannot be solved directly. Instead, we propose a heuristic algorithm to solve $\mathcal{P}_{\mathrm{r}}$, which works as following steps.

Step 1: Assume only one relay, e.g., i-th relay $i = 1, 2, \ldots, M^t$, is selected in relay assisted computation offloading, therefore $\boldsymbol{y}^t$ and $\boldsymbol{\rho}^t$ can be fixed as

$$y_k^t = \begin{cases} 1 & k = i \\ 0 & k \ne i, \end{cases} \tag{46}$$

and

$$\rho_k^t = \begin{cases} 1 & k = i \\ 0 & k \ne i, \end{cases} \tag{47}$$

respectively. In this case, $\mathcal{P}_{\mathrm{r}}$ can be simplified as

$$\mathcal{P}_{\mathrm{r},i}: \min_{p_{1,i}^t, p_{2,i}^t} \quad -\tilde{B}^t \cdot \frac{p_{1,i}^t L^t}{r_{1,i}^t} + V \cdot \left(\frac{L^t}{r_{1,i}^t} + \frac{L^t}{r_{2,i}^t}\right)$$

$$s.t. \quad \frac{p_{1,i}^t L^t}{r_{1,i}^t} \in [E_{\min}, E_{\max}] \tag{48}$$

$$\frac{L^t}{r_{1,i}^t} + \frac{L^t}{r_{2,i}^t} \le t_{\mathrm{d}} \tag{49}$$

$$0 \le p_{1,i}^t, p_{2,i}^t \le p_{\mathrm{trans}}^{\max}. \tag{50}$$

Since relay devices are energy sufficient, the i-th relay can transmit data to MEC servers with $p_{2,i}^t = p_{\mathrm{trans}}^{\max}$. Thus the transmission delay of i-th relay to MEC, $T_{2,i}^t$, is a known value, which can be calculated by (16). Hence, $\mathcal{P}_{\mathrm{r},i}$ can be further simplified

as

$$\mathcal{P}'_{\mathrm{r},i}: \min_{p^t_{1,i}} \quad -\tilde{B}^t \cdot \frac{p^t_{1,i} L^t}{r^t_{1,i}} + V \cdot \frac{L^t}{r^t_{1,i}} + C$$

$$s.t. \quad \frac{p^t_{1,i} L^t}{r^t_{1,i}} \in [E_{\min}, E_{\max}] \tag{51}$$

$$\frac{L^t}{r^t_{1,i}} \le t_{\mathrm{d}} - \frac{C}{V} \tag{52}$$

$$0 \le p^t_{1,i} \le p^{\max}_{\mathrm{trans}}, \tag{53}$$

where $C = VT^t_{2,i}$. It is found that $\mathcal{P}'_{\mathrm{r},i}$ is a direct offloading problem with execution time deadline $t_{\mathrm{d}} - C/V$, and we can easily obtain the optimal transmission power $p^{t,*}_{1,i}$ for $\mathcal{P}'_{\mathrm{r},i}$. Therefore the transmission delay with i-th relay device is

$$T^t_i = \frac{L^t_i}{r^t_{1,i}} + \frac{L^t_i}{r^t_{2,i}}, \tag{54}$$

where $L^t_i = L^t$.

Step 2: Repeat Step 1, we can obtain the transmission delay T^t_i of all the relays. Intuitively, if the relays with less transmission delay are selected in multi-relay assisted offloading, the total execution time T^t_{r} will be minimized. Therefore the relay selection problem can be formulated as

$$\min_{\boldsymbol{y}^t} \quad \sum_{i=1}^{M^t} T^t_i \cdot y^t_i$$

$$s.t. \quad (13),$$

which can be solved by greedy algorithm.

Step 3: Next, given $\boldsymbol{p}^{t*}$ and $\boldsymbol{y}^{t*}$, we obtain the optimal task splitting ratio $\boldsymbol{\rho}^{t*}$. It should be emphasized that in this step, as multiple relays are selected to assist computation offloading, the task is split into several parts and transmitted via different relay links. According to (19), T^t_{r} is the maximum execution time of all the relay links. To minimize T^t_{r}, we need to adjust the task splitting ratio such that

$$T^t_i = T^t_j, \quad \forall i, j \in [1, M^t],\ y^t_i = y^t_j = 1. \tag{55}$$

By using water filling algorithm [38], we can obtain $\boldsymbol{\rho}^{t*}$. Finally, we obtain the minimum execution time of relay assisted offloading T^t_{r} and energy consumption of mobile device E^t_{r}. All the steps are summarized in Algorithm 2. Note that T^t_{r} and E^t_{r} satisfy the execution time constraint (43) and the energy consumption constraint (44), respectively. The proof refers to Appendix A.

It should be mentioned that MRACO algorithm can also be applied to more general cases where there are multiple MEC servers and/or multiple mobile devices. In the case that there are multiple MEC servers (or BSs), since each mobile device can only be served by one server in each time slot, it is equivalent to the scenario considered in this paper and MRACO algorithm can be applied at each MEC server independently. On the other hand, in the case that there are multiple mobile devices to offload their tasks to a single MEC server, the mobile devices will first bid for the resources, e.g., relay devices and computation resources of the MEC server, and then apply MRACO algorithm to decide the optimal offloading strategies. However, the bidding strategy design is beyond the scope of this paper.

Algorithm 2: Multi-relay Task Assignment Algorithm.

Input: (L^t, t^t_{d}), h_t, mobile device position (a_0, b_0), base station position $(a_{\mathrm{m}}, b_{\mathrm{m}})$, relay devices positions (a_i, b_i), M^t,K^t,$\tilde{B}^t$,V

Output: $J^t_{\mathrm{r}}(\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t)$, $\boldsymbol{p}^t$,$\boldsymbol{y}^t$,$\boldsymbol{\rho}^t$

for $i = 1$ **to** M^t **do**
  $d^t_{1,i} \leftarrow \sqrt{(a_i - a_0)^2 + (b_i - b_0)^2}$;
  $d^t_{2,i} \leftarrow \sqrt{(a_i - a_m)^2 + (b_i - b_m)^2}$;
  calculate $p^t_{1,i}, p^t_{2,i}$ based on $\mathcal{P}'_{\mathrm{r},i}$;
  calculate T^t_i based on (54);
end
$M^t_{\mathrm{relay}} \leftarrow \min\{M^t, K^t\}$;
$\boldsymbol{y}^{t*} \leftarrow \arg \min_{\sum_{i=1}^{M^t} y^t_i = M^t_{relay}} \sum_{i=1}^{M^t} T^t_i \cdot y_i$;
$\boldsymbol{\rho}^t \leftarrow$ solution of Eq. (15) and Eq. (55) ;
for $i = 1$ **to** M^t **do**
  $L^t_i \leftarrow \rho^t_i L^t$;
  $T^t_{\mathrm{r}} \leftarrow \max\left[T^t_{\mathrm{r}}, (T^t_{1,i} + T^t_{2,i}) \cdot y^t_i\right]$;
  $E^t_{\mathrm{r}} \leftarrow E^t_{\mathrm{r}} + E^t_{1,i} \cdot y^t_i$;
end
$J^t_{\mathrm{r}}(\boldsymbol{p}^t, \boldsymbol{y}^t, \boldsymbol{\rho}^t) = -\tilde{B}^t \cdot E_{\mathrm{r}} + V \cdot T_{\mathrm{r}}$

D. Complexity and Optimality Analysis

Algorithm Complexity: In terms of algorithm complexity, the MRACO algorithm is composed of

1) Solving $\mathcal{P}_{\mathrm{m}}$: The CPU frequency and $J^t_{\mathrm{m}}(f^t)$ when execution locally can be calculated in $O(1)$.
2) Solving $\mathcal{P}_{\mathrm{s}}$: The bisection search algorithm [39] will be used 3 times when calculating transmission power. If the required accuracy of the solution is p_{req} (unit same with $p^{\max}_{\mathrm{trans}}$), the iteration number will less than $\log(p^{\max}_{\mathrm{trans}}/p_{\mathrm{req}})$. Then calculating $J^t_{\mathrm{m}}(f^t)$ in $O(1)$. So the totally time complexity is $O(\log(p^{\max}_{\mathrm{trans}}/p_{\mathrm{req}}))$.
3) Solving $\mathcal{P}_{\mathrm{r}}$: In algorithm 2, We calculate transmission power for M^t devices in which the time complexity is $O(M^t \log(p^{\max}_{\mathrm{trans}}/p_{\mathrm{req}}))$. Then we find the smallest M^t_{relay} transmission time by Quickselect [40] in $O(M^t)$. Finally calculating J^t_{r} in $O(M^t)$. The total time complexity of Algorithm 2 is $O(M^t \log(p^{\max}_{\mathrm{trans}}/p_{\mathrm{req}})) + O(M^t) = O(M^t \log(p^{\max}_{\mathrm{trans}}/p_{\mathrm{req}}))$.

Therefore, the time complexity of MRACO algorithm (Algorithm 1) is $O(M^t \log(p^{\max}_{\mathrm{trans}}/p_{\mathrm{req}}))$.

Optimality Analysis: Note that $\mathcal{P}_1$ is the original optimization problem, $\mathcal{P}_2$ is the transformed problem using the Lyapunov optimization technique. Let $\mathcal{P}^*_2$ denote the optimal solution of $\mathcal{P}_2$, it can be proved that [20], [41]

$$\mathcal{P}^*_2 \le \mathcal{P}^*_1 + \frac{K}{V}, \tag{56}$$

TABLE II
SIMULATION PARAMETERS

Parameters	Symbols	Default value
Number of relay devices	-	6
Path-loss constant	g_0	-40 dB
Reference distance	d_0	1 m
Size of the scene (square)	$L_{\mathrm{BS}} \times L_{\mathrm{BS}}$	180 m $\times$180 m
Effective switched capacitance	κ	10^{-28}
Time slot length	Δt	20 ms
Penalty factor	ϕ	20 ms
Bandwidth	ω	1 MHz
Noise power	σ	10^{-13} W
Maximum transmission power	$p_{\mathrm{trans}}^{\max}$	1 W
Maximum CPU frequency	$f_{\mathrm{CPU}}^{\max}$	1.5 GHz
Task size	L	1000 bits
Number of CPU cycles required to process one bit	X	5900 cycles / bit
Task execution deadline	t_{d}	20 ms
Mobility parameter	β	0.7
Mobility parameter	γ	0.5
Maximum energy consumption per task execution	$E_{\max}$	2 mJ
Minimum energy consumption per task execution	$E_{\min}$	0.02 mJ
Energy harvesting power	P_{H}	12 mW
Maximum number that the task can be split into	K^t	3

where $\mathcal{P}_1^*$ is the optimal solution of $\mathcal{P}_1$, and K is a constant value. This indicates that the solution of $\mathcal{P}_2$ can asymptotic achieve the optimal solution of the original problem $\mathcal{P}_1$ when $V \to +\infty$.

VI. PERFORMANCE EVALUATION

A. Simulation Setup

In the simulations, we assume that the range of a single base station is a square with a side equal to $L_{BS} = 180$ m, and the base station locates at the center. There are 6 relay devices and a mobile device in the system. In terms of the mobile device, we set the battery level $B^0 = 0$ at the initial time slot, and there is no computing task at this time slot. Then the mobile device will start to harvest energy e^0 and use them at the next time slot. We set the default average energy harvesting power as $P_{\mathrm{H}} = E_H^{\max}(2\Delta t)^{-1} = 12$ mW. E_{H}^t is uniformly distributed between 0 and $E_{\mathrm{H}}^{\max}$.

In general, $\kappa = 10^{-28}$, $\Delta t = \phi = 20$ ms, $\omega = 1$ Mhz, $\sigma = 10^{-13}$ W, $p_{\mathrm{trans}}^{\max} = 1$ W, $f_{\mathrm{CPU}}^{\max} = 1.5$ GHz, $X = 5900$ cycles per byte. $E_{\max} = 2$ mJ, $E_{\min} = 0.02$ mJ. The comuting task size $L^t = 10000$ bits. Moreover, the default execution time deadline $t_{\mathrm{d}}^t = 20$ ms, and the default task arrival rate is 0.6. The above experimental data settings refer to [20], [30], [42]. For convenience, the simulation parameters are listed in Table II.

To verify the effectiveness of the proposed MRACO algorithm, we compare its performance with two benchmark algorithms:

- The LODCO algorithm [20] can only execute the computation tasks locally or directly offload them to the MEC server, which cannot use the relay link for offloading execution.
- The LODCO-based cooperative offloading algorithm [14] is the single-relay assisted computation offloading (SRACO) algorithm, which can only have one relay device when offloading to the MEC server. The SRACO algorithm will select a surrounding device with the smallest transmission delay as the relay device.

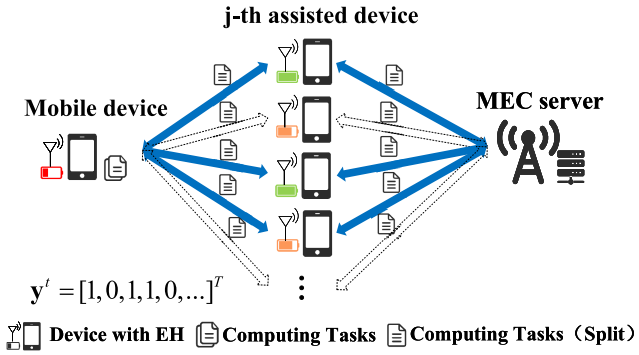


Fig. 2. The selection of relay devices.

B. Impact of K^t

We first change the maximum number that the task can be split into, K^t, from 0 to 5 and evaluate its impact on execution cost, execution time, drop rate, and relay mode rate. In particular, $K^t = 0$ means the mobile device is not allowed to use relay assisted offloading. The results are shown in Fig. 3, where the red and blue curves represents the execution time deadline t_d is set at 20 ms and 4 ms, respectively.

In Fig. 3(a), it can be seen that the execution cost is highest when the relay assisted offloading is not allowed. As the K^t increases, the execution cost decreases and tends to converge after $K^t = 3$. The tendency of execution time in Fig. 3(b) is similar to that of execution cost in Fig. 3(a). Fig. 3(c) shows that the drop rate decreases with K^t when $t_d = 4$ ms. However, when $t_d = 20$ ms, the drop rate remains unchanged and is very close to 0. This is because when $t_d = 20$ ms, even with $K^t = 0$, the execution time per task is around 5 ms, which is far less than 20 ms, as can be seen in Fig. 3(b). In Fig. 3(d), the relay mode rate first increases with K^t, and then converges after K^t exceeds 2.

The results indicates that we cannot improve the system performance by simply increasing K^t. The system performance becomes stable when the K^t exceeds a certain value. Moreover, a larger K^t will make offloading decisions more complex. Therefore, we set K^t at 3 in the following simulations.

C. Impact of Energy Harvesting Power

Battery energy level is an important parameter for mobile devices. To evaluate the performance of different algorithms for different battery energy level states, we will control the energy level of the battery by changing the energy harvesting power. When the energy harvesting power is small, the battery energy level will be relatively low, and the MEC-EH system will be in a poor state; on the contrary, when the energy level is relatively high, the battery will obtain sufficient energy to provide better performance.

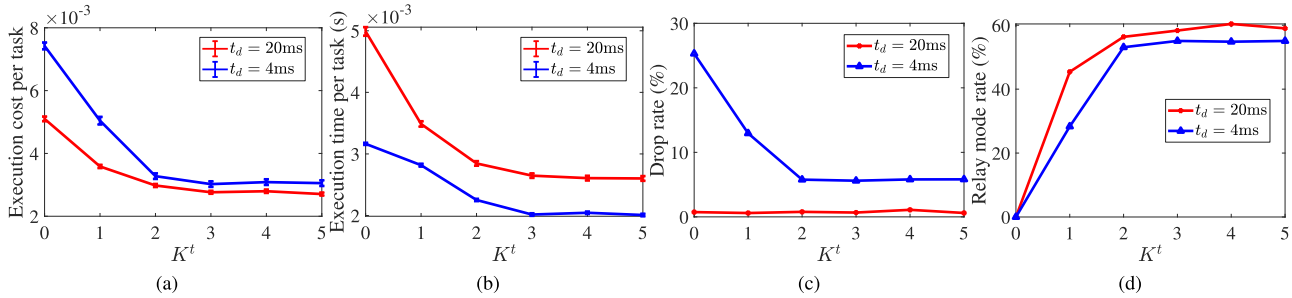


Fig. 3. Impact of K^t on (a) execution cost per task (b) execution time per task (c) drop rate (d) relay mode rate.

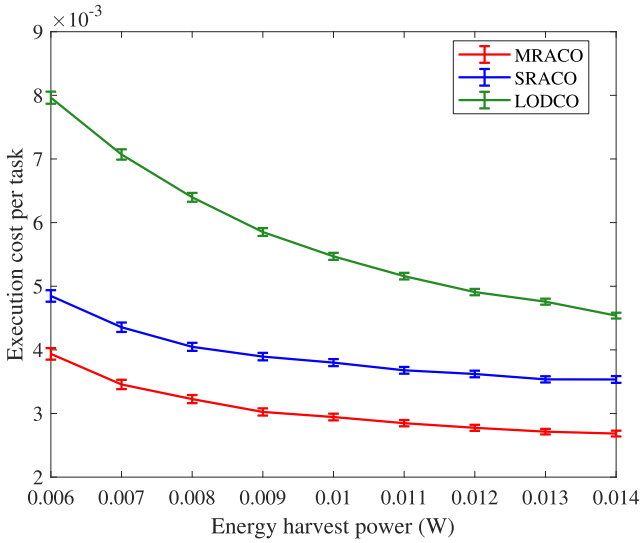


Fig. 4. Execution cost vs. energy harvesting power.

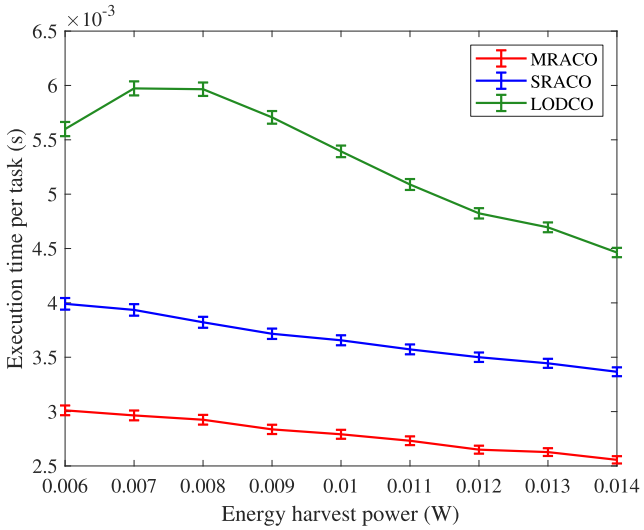


Fig. 5. Execution time vs. energy harvesting power.

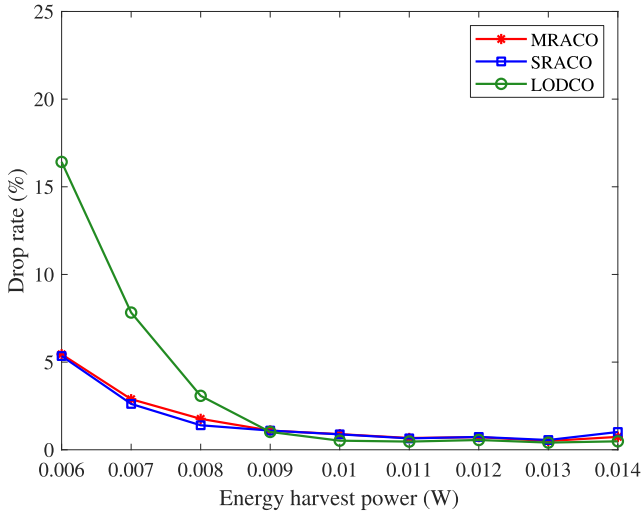


Fig. 6. Drop rate vs. energy harvesting power.

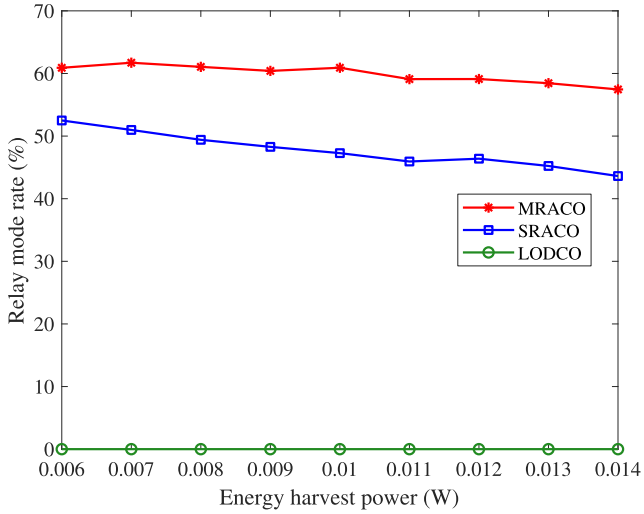


Fig. 7. Relay mode rate vs. energy harvesting power.

In this simulation experiment, we will control the energy harvesting power from 6 mW to 14 mW and keep all of the other parameters the same as the default values. The simulation results of three different algorithms are shown in Fig. 4 to Fig. 7.

It can be determined from Fig. 4 that when the energy harvesting power is small, the MEC-EH system status of the mobile device will be poor and the task execution cost will be high. As the energy harvesting power increases, the battery energy level increases, and the execution cost gradually decreases. Among the three algorithms, the MRACO algorithm achieved the lowest

execution cost. In particular, when the battery energy level is low, the execution cost is reduced by more than 50% compared to the LODCO algorithm and by more than 18% compared to the SRACO algorithm.

As can be determined from Fig. 5, with the increase of energy harvesting power, the execution time first increases slightly and then gradually decreases. This is because when the energy level is low, many computing tasks cannot be completed normally and are dropped, resulting in task execution failure (in which case the execution time is zero). Thus, the completion of this part of the task will increase the execution time as the energy level increases. As device battery levels increase continuously, execution time will gradually decrease. Among the three algorithms, the time required by the MRACO algorithm is always lower than those of the other two algorithms. When the energy collection power is 8 mW, it is 50% lower than the LODCO algorithm and more than 23% lower than the SRACO algorithm. When the energy level increases, the gap will decrease, but the MRACO algorithm is still better than the other two algorithms.

In Fig. 6, when the energy level is relatively low, many execution tasks are dropped because they cannot be executed. As the energy level increases, the drop rate will gradually decrease. The MRACO algorithm proposed in this paper has a similar drop rate with the SRACO algorithm and is always better than the LODCO algorithms when the energy is relatively low. For example, when the energy harvesting power is 6 mW, the drop rate is only 5.1%, while that of the LODCO algorithm is more than 16%. As the energy level increases, the drop rates of all three algorithms gradually converge to less than 1%. It can be determined that the MRACO algorithm performs better when the energy state is poor.

Fig. 7 shows the proportion of the relay mode used during task execution. It can be found that MRACO algorithm increases the probability of the relay mode compared to SRACO algorithm. This is also the main reason for the gap between the performances of these two algorithms. According to the comparison experiments of energy harvest power, the MRACO algorithm offers stronger advantages than the other two algorithms, especially when the energy level is relatively low.

D. Impact of Task Arrival Rate

The task arrival rate directly reflects the degree of task congestion. In the MEC-EH system model, it mainly impacts the energy consumption. Since the mobile device with EH is expected to achieve energy self-sufficiency, the arrival of tasks will challenge the energy level. Eventually, they will lead to an increase in the time and cost of execution tasks.

In this part of the simulation experiment, we will control the task arrival rate from 10% to 100% per time slot, keeping the other conditions unchanged. The results of three different algorithms are shown in Fig. 8 to Fig. 11.

It can be determined from Fig. 8 that the overall trend is that as the task arrival rate increases, the execution cost continues to increase. The MRACO algorithm proposed in this paper can effectively reduce the task execution cost. When the task arrival rate is 100%, that is, each time slot has a computation task arrival,

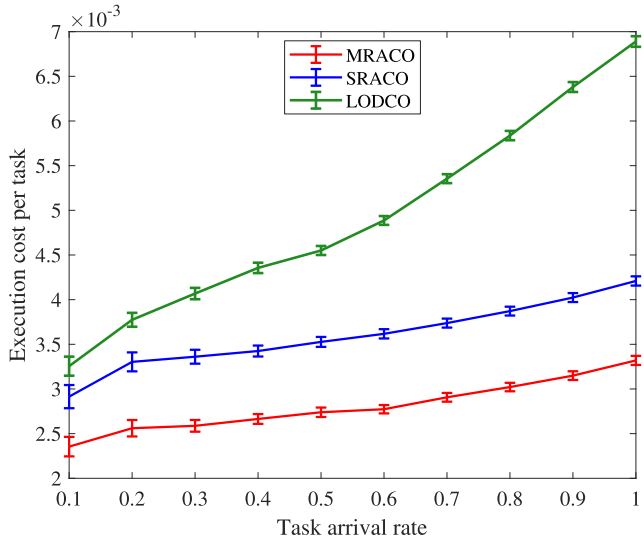


Fig. 8. Execution cost vs. task arrival rate.

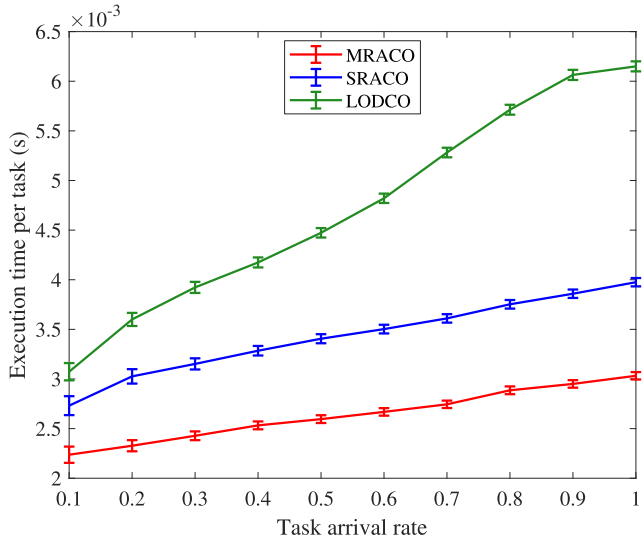


Fig. 9. Execution time vs. task arrival rate.

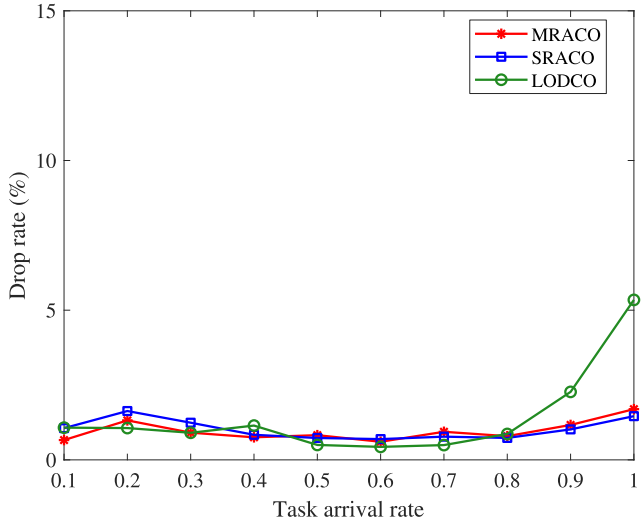


Fig. 10. Drop rate vs. task arrival rate.

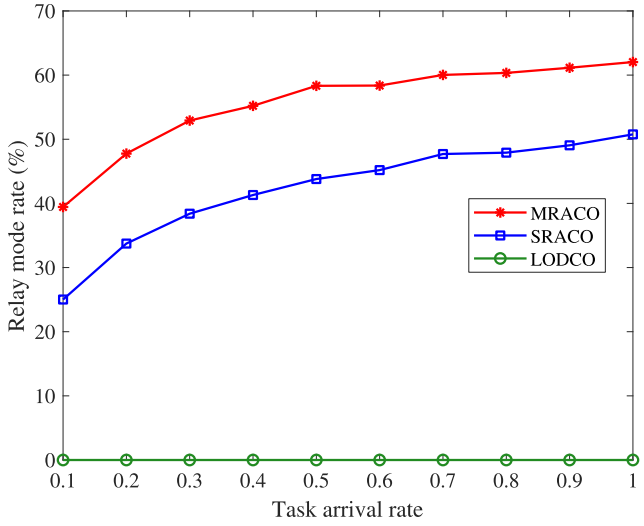


Fig. 11. Relay mode rate vs. task arrival rate.

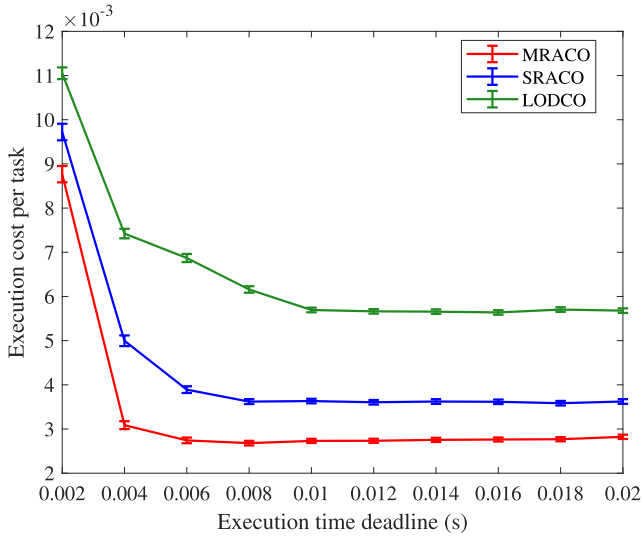


Fig. 12. Execution cost vs. execution time deadline.

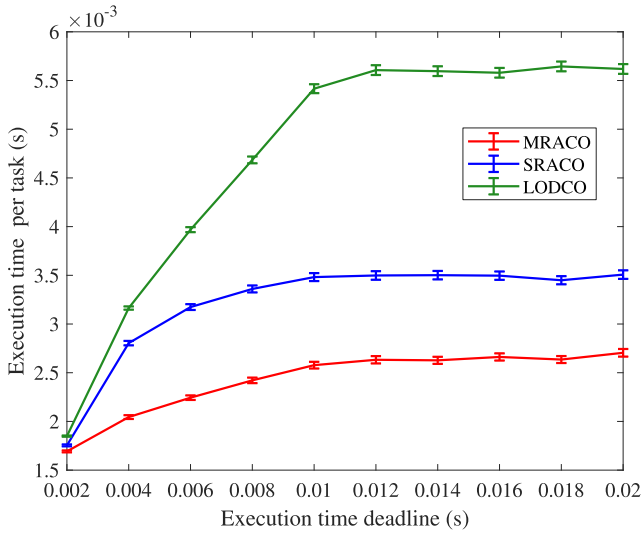


Fig. 13. Execution time vs. execution time deadline.

the task execution cost required by the MRACO algorithm is approximately 52% less than that of the LODCO algorithm, and 22% less than that of the SRACO algorithm. Obviously, under high load conditions, the MRACO algorithm can achieve better performance than other algorithms.

As can be determined from Fig. 9, as the task arrival rate becomes larger, the overall task execution time continues to increase. The MRACO algorithm proposed in this paper is superior to the other two algorithms in all states of arrival rate, and as the task arrival rate increases, the advantages become increasingly larger, which is similar to the results of task execution cost.

E. Energy Computation Analysis

In Fig. 10, as the task arrival rate increases, the drop rate of the task will not greatly change. Only when the task drop rate is very large will the drop rate increases. In terms of the energy-saving advantages of relay-assisted offloading, both the MRACO algorithm and the SRACO algorithm are better than the LODCO algorithm, which cannot perform relay-assisted offloading.

It can be determined from Fig. 11 that the relay mode probability of the MRACO algorithm is higher than that of the SRACO algorithm by up to 14%, which leads to a huge gap between the two algorithms.

In general, the MRACO algorithm can still perform better than the other two algorithms in the case of a high task execution rate and exhibits good system performance. These experiments further illustrate the advantages of multi-relay assistance in poor environments.

F. Impact of Execution Time Deadline

Time-sensitive applications are typically executed on mobile devices and have high requirements for execution time. In this subsection, we will control the deadline of the task from 2 ms to 20 ms. If the task execution time is longer than the deadline, the task will be dropped, resulting in a large cost. The results of the MEC-EH system are shown in Fig. 12 to Fig. 15.

It can be determined from Fig. 12 that the execution cost of the LODCO algorithm gradually decreases as the task deadline increases. Similarly, SRACO and MRACO algorithms, which can perform relay mode execution, also have large costs when the execution time deadline is small. However, as the execution time deadline increases, there will be an immediate decline, and then the execution costs will be maintained at a low level. Among them, MRACO algorithm has a lower execution cost than SRACO algorithm and is more suitable for tasks with higher time requirements.

Fig. 13 shows the change in execution time as the task execution time deadline increases. Due to time constraints, the execution time gradually increased and eventually stabilized. The MRACO algorithm requires less time than the other two algorithms. This is because the multi-relay and task split offloading greatly reduce the execution time. As can be determined from

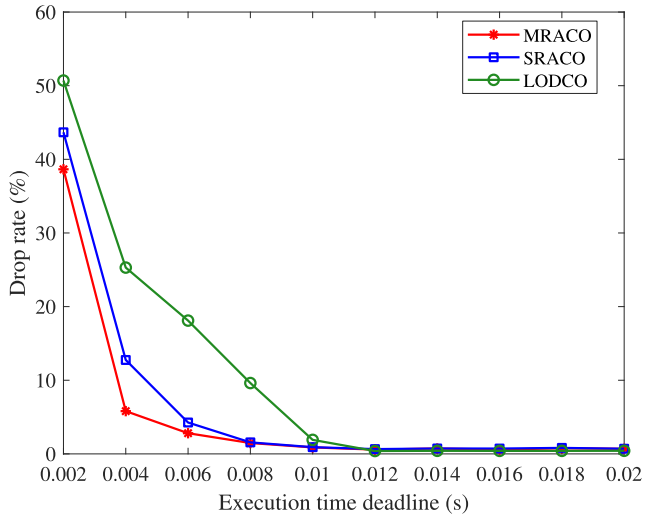


Fig. 14. Drop rate vs. execution time deadline.

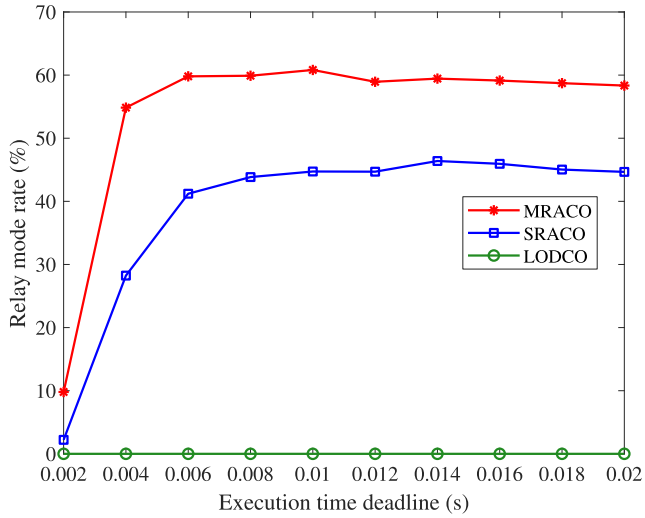


Fig. 15. Relay mode rate vs. execution time deadline.

Fig. 13, compared to the LODCO algorithm and SRACO algorithm, the MRACO algorithm can reduce up to 51% and 22%, respectively. The MRACO algorithm still presents advantages under higher time requirements.

It can be determined from Fig. 14 that there is generally a high task drop rate when the task execution time deadline is small. When the time deadline is small (from 2 ms to 6 ms), the MRACO algorithm has a lower drop rate than the MRACO algorithm. As the time deadline increases, the MRACO algorithm and SRACO algorithms with relay mode execution strategies decrease rapidly, but the LODCO algorithm decreases slowly.

In Fig. 15, as the deadline increases, the proportion of the relay mode will increase slightly, and then slowly decline. This is because other task execution strategies begin to become available, and the relay mode is no longer the only available strategy. The relay mode rate of the MRACO algorithm is always higher than that of the SRACO algorithm, and the relay mode execution strategy will make more contributions when the deadline is small.

In general, the MRACO algorithm still offers advantages when handling time-sensitive tasks, and it has lower execution time, drop rate and execution costs.

The energy computation is not an optimal object in this paper as shown in (24). But it is an important factor and a constraint condition in $\mathcal{P}_1$. Therefore, it is necessary to observe the energy computation performance in different execution strategies.

In Fig. 16(a), we change the energy harvest power while other parameters keep the default. With the increase of energy harvest power, the energy consumption of each execution strategy raises. This is reasonable because we do not minimize energy consumption. What we guarantee is that the battery level will be stable around a certain range so that the mobile device can ensure energy self-sufficiency and its battery will not be depleted. On this basis, energy will be used to increase the CPU frequency and transmission power, which results in a lower cost as shown in Fig. 4.

Similar to Fig. 16(a), we change task arrival rate and execution time deadline individually and keep other parameters default. In Fig. 16(b), As the task arrival rate increases, there is less energy consumption for each task. That's because lower task arrival rate means more energy available for each task. When tasks become intensive, the battery will be used to execute more tasks so that the available energy per task is lower. Lower energy consumption means higher execution cost, as shown in Fig. 8.

In Fig. 16(c), however, energy consumption does not visibly vary with the execution time deadline. The reason is that these strategies try to execute each task at minimum cost (lower execution time and lower average drop rate) despite being limited by a loose time deadline. The only effect of a low execution time deadline is that some tasks will be dropped (as shown in Fig. 14) and do not consume energy. Other tasks will consume more energy so that they have lower execution time and will not be dropped. Therefore, when the execution time deadline is 0.002 s, the energy consumption is slightly lower than other results in Fig. 16(c).

In terms of comparison of different execution strategies, the energy consumption of the MRACO algorithm is lower than other strategies. Although we do not minimize the energy consumption, the energy consumption will be saved when the mobile device offloads tasks via relays. In this way, the mobile device only consumes a part of transmission energy, the others are consumed by relay devices.

VII. Conclusion

In this paper, we considered a multi-relay assisted MEC-EH system in which the mobile devices can offload tasks to the MEC server with the help of multiple neighboring nodes. We introduce execution cost, which incorporates both the execution time and task failure rate, as the performance metric. We formulated the execution cost minimization problem as a high-dimensional Markov decision problem. We also proposed the MRACO algorithm to dynamically select the optimal offloading strategy at each time slot, i.e., whether to execute the task locally, directly offload the task to the MEC server, offload it to the MEC server via multiple relays, or just drop this task. A series

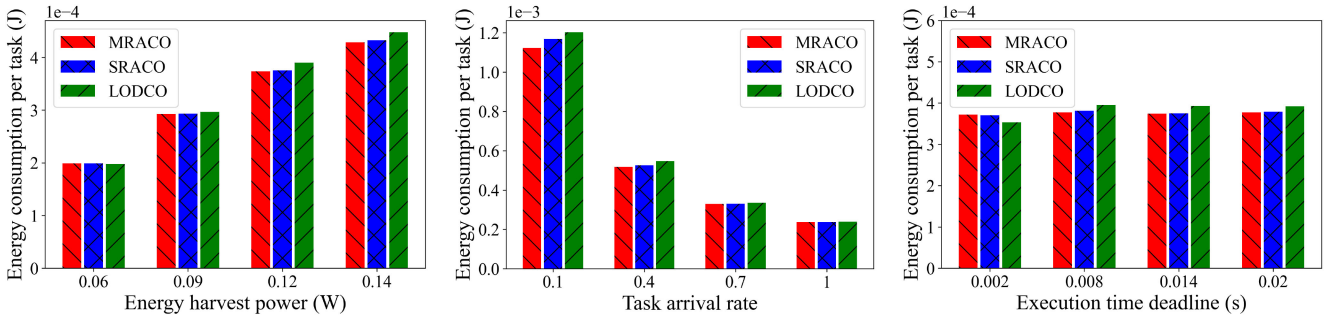


Fig. 16. Energy consumption per task vs. (a) energy harvest power (b) task arrival rate (c) execution time deadline.

of simulations demonstrated that the proposed algorithm can significantly improve the system performance; e.g., the average task execution cost can be reduced by up to 45% as compared with the state-of-the-art solution. This outcome verifies that with multi-relay assisted cooperative offloading, the tasks have a higher probability to be offloaded to the MEC server to enjoy the benefits of MEC. In this paper, it is assumed that all the neighboring devices are willing to participate in the offloading process. However, in practice, the problem of how to design a proper incentive mechanism to encourage the neighboring devices to act as relay nodes is a critical issue. This issue is left for future study.

Appendix A
Proof That the Constraints of T_{r}^t and E_{r}^t is Satisfied

A. Proof $T_{\mathrm{r}}^t \le t_{\mathrm{d}}$

According to the solution of $\mathcal{P}'_{\mathrm{r}}$, we can calculate the execution time $T'^t_{1,i} = \frac{L^t}{r^t_{1,i}}$ satisfying $T'^t_{1,i} \le t_{\mathrm{d}} - \frac{C}{V}$.

Because $T_{\mathrm{r}}^t = \max_{i\in[1,M^t]}[(T^t_{1,i} + T^t_{2,i}) \cdot y^t_i]$ and $T^t_{2,i}$ is a certain value $\frac{C}{V}$, we just need to proof $\max_{i\in[1,M^t]}[T^t_{1,i} \cdot y^t_i] \le t_{\mathrm{d}} - \frac{C}{V}$. Therefore we have

$$\begin{aligned}\max_{i\in[1,M^t]}\left[T^t_{1,i}\cdot y^t_i\right] &= \max_{i\in[1,M^t]}\left[T'^t_{1,i}\cdot\rho^t_i\cdot y^t_i\right]\\ &\le T'^t_{1,\max}\max_{i\in[1,M^t]}\left[\rho^t_i\cdot y^t_i\right]\\ &\le T'^t_{1,\max}\end{aligned}$$

where $T'^t_{1,\max} = \max_{i\in[1,M^t]} T'^t_{1,i} \le t_{\mathrm{d}} - \frac{C}{V}$.

So that $\max_{i\in[1,M^t]}[T^t_{1,i}\cdot y^t_i] \le t_{\mathrm{d}} - \frac{C}{V}$, and we have $T_{\mathrm{r}}^t \le t_{\mathrm{d}}$.

B. Proof $E_{\mathrm{r}}^t \in [E_{\min}, E_{\max}]$

According to the solution of $\mathcal{P}'_{\mathrm{r}}$, we can calculate the energy consumption $E'^t_{1,i} = \frac{p^t_{1,i}L^t}{r^t_{1,i}}$, which satisfies $E'^t_{1,i} \in [E_{\min}, E_{\max}]$.

In $\mathcal{P}'_{\mathrm{r}}$, however, we transfer all the tasks in each relay link, and the task will be split in the final execution. So we have

$$\begin{aligned}E_{\mathrm{r}}^t = \sum_{i=1}^{M^t}\left[E^t_{1,i}\cdot y^t_i\right] &= \sum_{i=1}^{M^t}\left[E'^t_{1,i}\cdot\rho^t_i\cdot y^t_i\right]\\ &\ge \sum_{i=1}^{M^t}\left[E'^t_{1,\min}\cdot\rho^t_i\cdot y^t_i\right]\\ &= E'^t_{1,\min}\end{aligned}$$

where $E'^t_{1,\min} = \min_{i\in[1,M^t]} E'^t_{1,i} \ge E_{\min}$, therefore we have $E_{\mathrm{r}}^t \ge E_{\min}$.

Similarly, we can proof $E_{\mathrm{r}}^t \le E_{\max}$ as follows:

$$\begin{aligned}E_{\mathrm{r}}^t = \sum_{i=1}^{M^t}\left[E^t_{1,i}\cdot y^t_i\right] &= \sum_{i=1}^{M^t}\left[E'^t_{1,i}\cdot\rho^t_i\cdot y^t_i\right]\\ &\le \sum_{i=1}^{M^t}\left[E'^t_{1,\max}\cdot\rho^t_i\cdot y^t_i\right]\\ &= E'^t_{1,\max}\end{aligned}$$

where $E'^t_{1,\max} = \max_{i\in[1,M^t]} E'^t_{1,i} \le E_{\max}$, therefore we have $E_{\mathrm{r}}^t \le E_{\max}$.

In conclusion we have $E_{\mathrm{r}}^t \in [E_{\min}, E_{\max}]$.

References

[1] T. K. Rodrigues, K. Suto, H. Nishiyama, J. Liu, and N. Kato, "Machine learning meets computation and communication control in evolving edge and cloud: Challenges and future perspective," *IEEE Commun. Surv. Tut.*, vol. 22, no. 1, pp. 38–67, Sep. 2020.

[2] T. Qiu, J. Liu, W. Si, and D. O. Wu, "Robustness optimization scheme with multi-population co-evolution for scale-free wireless sensor networks," *IEEE/ACM Trans. Netw.*, vol. 27, no. 3, pp. 1028–1042, Jun. 2019.

[3] N. Chen, T. Qiu, X. Zhou, K. Li, and M. Atiquzzaman, "An intelligent robust networking mechanism for the internet of things," *IEEE Commun. Mag.*, vol. 57, no. 11, pp. 91–95, Nov. 2019.

[4] K. Gai, M. Qiu, H. Zhao, L. Tao, and Z. Zong, "Dynamic energy-aware cloudlet-based mobile cloud computing model for green computing," *J. Netw. Comput. Appl.*, vol. 59, pp. 46–54, Jan. 2016.

[5] J. Zhao, Q. Li, Y. Gong, and K. Zhang, "Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, p. 7944–7956, Aug. 2019.

[6] T. G. Rodrigues, K. Suto, H. Nishiyama, and N. Kato, "Hybrid method for minimizing service delay in edge cloud computing through VM migration and transmission power control," *IEEE Trans. Comput.*, vol. 66, no. 5, pp. 810–819, May 2017.

[7] T. Qiu, R. Qiao, and D. O. Wu, "EABS: An event-aware backpressure scheduling scheme for emergency Internet of Things," *IEEE Trans. Mobile Comput.*, vol. 17, no. 1, pp. 72–84, Jan. 2018.

[8] K. Gai, K. Xu, Z. Lu, M. Qiu, and L. Zhu, "*Fusion of cognitive wireless networks and edge computing*," *IEEE Wireless Commun.*, vol. 26, no. 3, pp. 69–75, Jul. 2019.

[9] T. Qiu, B. Li, X. Zhou, H. Song, I. Lee, and J. Lloret, "A novel shortcut addition algorithm with particle swarm for multisink Internet of Things," *IEEE Trans. Ind. Informat.*, vol. 16, no. 5, pp. 3566–3577, Jun. 2020.

[10] A. Toor, A. Wen, F. Maksimovic, A. M. Gaikwad, K. S. Pister, and A. C. Arias, "Stencil-printed lithium-ion micro batteries for IoT applications," *Nano Energy*, vol. 82, Apr. 2021, Art. no. 105666.

[11] Y. Mao, Y. Luo, J. Zhang, and K. B. Letaief, "Energy harvesting small cell networks: Feasibility, deployment, and operation," *IEEE Commun. Mag.*, vol. 53, no. 6, pp. 94–101, Jun. 2015.

[12] M. Min, L. Xiao, Y. Chen, P. Cheng, D. Wu, and W. Zhuang, "Learning-based computation offloading for IoT devices with energy harvesting," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1930–1941, Jan. 2019.

[13] W. Zhang, Y. Wen, K. Guan, D. Kilper, H. Luo, and D. O. Wu, "Energy-optimal mobile cloud computing under stochastic wireless channel," *IEEE Trans. Wireless Commun.*, vol. 12, no. 9, pp. 4569–4581, Aug. 2013.

[14] M. Li, T. Chen, J. Zeng, X. Zhou, K. Li, and H. Qi, "D2D-assisted computation offloading for mobile edge computing systems with energy harvesting," in *Proc. 20th Int. Conf. Parallel Distrib. Comput., Appl. Technol.*, 2019, pp. 90–95.

[15] J. Liu, Y. Mao, J. Zhang, and K. B. Letaief, "Delay-optimal computation task scheduling for mobile-edge computing systems," in *Proc. IEEE Int. Symp. Inform. Theory*, 2016, pp. 1451–1455.

[16] Y. Wu, L. P. Qian, K. Ni, C. Zhang, and X. Shen, "Delay-minimization nonorthogonal multiple access enabled multi-user mobile edge computation offloading," *IEEE J. Sel. Topics Signal Process.*, vol. 13, no. 3, pp. 392–407, Jun. 2019.

[17] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Nov. 2019.

[18] J. Xu and S. Ren, "Online learning for offloading and autoscaling in renewable-powered mobile edge computing," in *Proc. IEEE Glob. Commun. Conf.*, 2016, pp. 1–6.

[19] A. Mehrabi, M. Siekkinen, and A. Ylä-Jääski, "Energy-aware QoE and backhaul traffic optimization in green edge adaptive mobile video streaming," *IEEE Trans. Green Commun. Netw.*, vol. 3, no. 3, pp. 828–839, May 2019.

[20] Y. Mao, J. Zhang, and K. B. Letaief, "Dynamic computation offloading for mobile-edge computing with energy harvesting devices," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3590–3605, Sep. 2016.

[21] L. Huang, S. Bi, and Y.-J. A. Zhang, "Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 19, no. 11, pp. 2581–2593, Nov. 2020.

[22] P. Gandotra and R. K. Jha, "Device-to-device communication in cellular networks: A survey," *J. Netw. Comput. Appl.*, vol. 71, pp. 99–117, Aug. 2016.

[23] J. N. Laneman, D. N. C. Tse, and G. W. Wornell, "Cooperative diversity in wireless networks: Efficient protocols and outage behavior," *IEEE Trans. Inf. Theory*, vol. 50, no. 12, pp. 3062–3080, Nov. 2004.

[24] Y. Cao, T. Jiang, and C. Wang, "Cooperative device-to-device communications in cellular networks," *IEEE Wireless Commun.*, vol. 22, no. 3, pp. 124–129, Jul. 2015.

[25] M. Yao, L. Chen, T. Liu, and J. Wu, "Energy efficient cooperative edge computing with multi-source multi-relay devices," in *Proc. IEEE 21st Int. Conf. High Perform. Comput. Commun.; IEEE 17th Int. Conf. Smart City; IEEE 5th Int. Conf. Data Sci. Syst.*, 2019, pp. 865–870.

[26] Y. Li, G. Xu, K. Yang, J. Ge, P. Liu, and Z. Jin, "Energy efficient relay selection and resource allocation in D2D-enabled mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 15800–15814, Dec. 2020.

[27] Z. Zhao *et al.*, "A novel framework of three-hierarchical offloading optimization for MEC in industrial IoT networks," *IEEE Trans. Ind. Informat.*, vol. 16, no. 8, pp. 5424–5434, Aug. 2020.

[28] Y. Zhuang, X. Li, H. Ji, and H. Zhang, "Optimization of mobile MEC offloading with energy harvesting and dynamic voltage scaling," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2019, pp. 1–6.

[29] J. Zhang, J. Du, Y. Shen, and J. Wang, "Dynamic computation offloading with energy harvesting devices: A hybrid-decision-based deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9303–9317, Jun. 2020.

[30] X. Ge, J. Ye, Y. Yang, and Q. Li, "User mobility evaluation for 5G small cell networks based on individual mobility model," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 3, pp. 528–541, Feb. 2016.

[31] K. C. of Arts, T. Science, M. SylvesterJeyaraj, and M. Subadra, "A study on dynamic source routing in ad hoc wireless networks," *Int. J. Eng. Trends Technol.*, vol. 8, no. 7, pp. 401–410, Feb. 2014.

[32] W. Dou, W. Tang, B. Liu, X. Xu, and Q. Ni, "Blockchain-based mobility-aware offloading mechanism for fog computing services," *Comput. Commun.*, vol. 164, pp. 261–273, Dec. 2020.

[33] T. D. Burd and R. W. Brodersen, "Processor design for portable systems," *J. VLSI Signal Process. Syst. Signal, Image Video Technol.*, vol. 13, no. 2-3, pp. 203–221, Aug. 1996.

[34] F. Zhou and R. Q. Hu, "Computation efficiency maximization in wireless-powered mobile edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 19, no. 5, pp. 3170–3184, Feb. 2020.

[35] C. Zhan, H. Hu, X. Sui, Z. Liu, and D. Niyato, "Completion time and energy optimization in the UAV-enabled mobile-edge computing system," *IEEE Internet Things J.*, vol. 7, no. 8, pp. 7808–7822, May 2020.

[36] W. Hong *et al.*, "mmWave 5 G NR cellular handset prototype featuring optically invisible beamforming antenna-on-display," *IEEE Commun. Mag.*, vol. 58, no. 8, pp. 54–60, Sep. 2020.

[37] S. Q. Zhang, J. Lin, and Q. Zhang, "Adaptive distributed convolutional neural network inference at the network edge with ADCNN," in *Proc. 49th Int. Conf. Parallel Process.*, 2020, pp. 1–11.

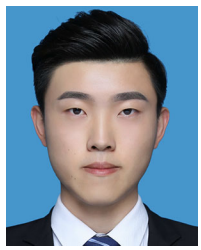
[38] J. Rao, H. Feng, C. Yang, Z. Chen, and B. Xia, "Optimal caching placement for D2D assisted wireless caching networks," in *Proc. IEEE Int. Conf. Commun.*, 2016, pp. 1–6.

[39] K. Sikorski, "Bisection is optimal," *Numerische Math.*, vol. 40, no. 1, pp. 111–117, Feb. 1982.

[40] C. A. Hoare, "Algorithm 65: Find," *Commun. ACM*, vol. 4, no. 7, pp. 321–322, Jul. 1961.

[41] M. J. Neely, "Stochastic network optimization with application to communication and queueing systems," *Synth. Lectures Commun. Netw.*, vol. 3, no. 1, pp. 1–211, Sep. 2010.

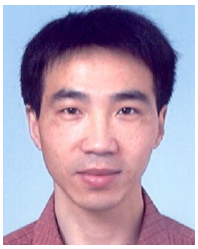
[42] G. R. MacCartney, T. S. Rappaport, and A. Ghosh, "Base station diversity propagation measurements at 73 GHz millimeter-wave for 5G coordinated multipoint (CoMP) analysis," in *Proc. IEEE Globecom Workshops*, 2017, pp. 1–7.



Molin Li received the bachelor's degree in communication engineering from the Dalian University of Technology, Dalian, China, in 2019. He is currently working toward the master's degree in computer science and technology from Tianjin University, Tianjin, China. His current research focuses on mobile edge computing.

Xiaobo Zhou (Senior Member, IEEE) received the B.Sc. degree in electronic information science and technology from the University of Science and Technology of China, Hefei, China, the M.E. degree in computer application technology from Graduate University of Chinese Academy of Science, Beijing, China, and the Ph.D. degree in information science from School of Information Science, Japan Advanced Institute of Science and Technology, Ishikawa, Japan, in 2007, 2010, and 2013, respectively. From April 2014 to March 2015, he was a Researcher with the Department of Communications Engineering, University of Oulu, Oulu, Finland. He is currently an Associate Professor with the School of Computer Science and Technology, Tianjin University, Tianjin, China. His research interests include cooperative wireless communications, wireless networks, data center networks, vehicular networks, and mobile edge computing.

Tie Qiu (Senior Member, IEEE) received the Ph.D. degree in computer science from the Dalian University of Technology, Dalian, China, in 2012. He is currently a Full Professor with the School of Computer Science and Technology, Tianjin University, Tianjin, China. Prior to this position, he was an Associate Professor in 2013 with the School of Software, Dalian University of Technology. He was a Visiting Professor of electrical and computer engineering with Iowa State University, Ames, IA, USA. He has authored or coauthored nine books, more than 150 scientific papers in international journals and conference proceedings, such as the IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, and *IEEE Communications Magazine*. He is an Associate Editor for the IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, and the Area Editor of the *Ad Hoc Networks* (Elsevier). He is the General Chair, Program Chair, Workshop Chair, Publicity Chair, Publication Chair of a number of international conferences. He is a Senior Member of China Computer Federation and a Senior Member of ACM.



Qinglin Zhao (Senior Member, IEEE) received the Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China, in 2005. From May 2005 to August 2009, he was a Postdoctoral Researcher with The Chinese University of Hong Kong, Hong Kong, and Hong Kong University of Science and Technology, Hong Kong. Since September 2009, he has been with the Faculty of Information Technology, Macau University of Science and Technology, Cotai, China, and he is currently a Professor. His research interests include blockchain and decentralization computing, machine learning and its applications, Internet of Things, wireless communications and networking, cloud or fog computing, and software-defined wireless networking.

Keqiu Li (Senior Member, IEEE) received the bachelor's and master's degrees from the Department of Applied Mathematics, Dalian University of Technology, Dalian, China, in 1994 and 1997, respectively, and the Ph.D. degree from the Graduate School of Information Science, Japan Advanced Institute of Science and Technology, Nomi, Japan, in 2005. He is currently a Full Professor, the Dean of the College of Intelligence and Computing, Tianjin University, Tianjin, China. He keeps working on the topics of mobile computing, data center, and cloud computing. He has authored or coauthored more than 150 papers on prestigious journals or conferences, including the IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON MOBILE COMPUTING, INFOCOM, and ICNP. He was the recipient of the National Science Foundation for Distinguished Young Scholars of China.